SCHOOL OF ARCHITECTURE, COMPUTING & ENGINEERING

DISSERTATION PROPOSAL

Big Data Log Anomaly Detection System Using Machine Learning

Programme	MSc in Artificial Intelligence
Student Name	Subhan Jameel
Student ID	U2926092
Year / Term	2025/26 | Term 3
Supervisor Name	Abdul Qadoos
Module Code	CN7000

1. Project Summary

This project develops a machine learning-based log anomaly detection system designed to automatically identify abnormal patterns and failure events in large-scale distributed system logs. System administrators, DevOps engineers, and site reliability engineers (SREs) operating high-availability infrastructure can benefit from automated anomaly detection to reduce mean time to detection (MTTD), prevent cascading failures, and improve overall system reliability.

Modern distributed systems generate massive volumes of structured and unstructured log data at rates that exceed human capacity for manual inspection. Traditional rule-based log monitoring approaches rely on fixed thresholds and manually authored alert patterns that fail to detect novel failure modes, produce high false-positive rates, and require constant maintenance as system behaviour evolves. These limitations result in missed anomalies, alert fatigue, and delayed incident response.

This system applies machine learning algorithms — including Isolation Forest, LSTM Autoencoders, Random Forest, and Logistic Regression — to the BGL (Blue Gene/L) supercomputer log dataset sourced from the LogHub official research repository (https://github.com/logpai/loghub/tree/master/BGL). Models are trained on structured log sequences capturing system event types, severity levels, component identifiers, message templates, and temporal event patterns over time.

System operators access the platform through a monitoring dashboard where they configure log stream endpoints and receive real-time anomaly alerts, anomaly confidence scores, the specific log sequences contributing to each anomaly decision, and SHAP (SHapley Additive exPlanations) feature

attributions for interpretable anomaly explanations. Model performance is evaluated using Precision, Recall, F1-score, Area Under the ROC Curve (AUC-ROC), and detection latency. The system is expected to achieve an F1-score above 0.90 on the BGL dataset benchmark and a false positive rate below 5%.

1b. Abstract

Large-scale distributed systems and high-performance computing clusters generate continuous streams of log data that contain critical signals for system health monitoring and fault diagnosis. Manual inspection of these logs is computationally infeasible at scale, and rule-based monitoring systems fail to detect novel or subtle anomalies that do not match predefined patterns. This proposal presents the design and implementation of a machine learning-based big data log anomaly detection system capable of automatically identifying anomalous log sequences associated with hardware faults, software failures, and system instability.

The system applies four candidate machine learning algorithms — Logistic Regression (baseline), Random Forest, Isolation Forest, and LSTM Autoencoders — trained on the BGL (Blue Gene/L) supercomputer log dataset obtained from the LogHub benchmark repository. The BGL dataset contains over four million log messages labelled with alert and non-alert categories, derived from IBM's Blue Gene/L supercomputer at Lawrence Livermore National Laboratory. Log parsing is performed using the Drain log parsing algorithm to extract structured event templates from raw unstructured log messages, enabling consistent feature extraction across diverse log formats. Temporal sequence modelling captures the ordering and co-occurrence patterns of log events that precede system failures.

A SHAP-based explainability layer provides per-prediction feature importance attributions, identifying which log events and temporal patterns most strongly signal anomalous system behaviour. Evaluation benchmarks all models against a rule-based baseline using Precision, Recall, F1-score, AUC-ROC, and detection latency metrics. Deliverables include a deployable log monitoring dashboard, a REST API for real-time anomaly scoring, complete source code on GitHub, and a comparative evaluation report. This work contributes to the growing body of research on intelligent log analysis by demonstrating that interpretable machine learning can operationally replace brittle rule-based monitoring with adaptive, data-driven anomaly detection.

1c. Problem Statement

Modern distributed systems, cloud platforms, and high-performance computing clusters generate log data at a scale and velocity that fundamentally exceeds human capacity for manual inspection. A single production system can produce millions of log entries per day across hundreds of nodes, components, and services. System failures, hardware faults, and security incidents are often preceded by subtle anomalous log patterns that unfold over time, making point-in-time threshold alerts insufficient for early fault detection.

The specific technical challenge addressed by this project is the inability of existing rule-based log monitoring systems to automatically detect novel, previously unseen anomaly patterns without extensive manual rule authoring. Current production monitoring tools such as Splunk, ELK Stack, and Nagios rely on static pattern matching and threshold-based alerting that produces high false-positive rates, suffers from alert fatigue, and cannot adapt to evolving system behaviour without continuous manual maintenance. The absence of learned anomaly models that can generalise across diverse log event patterns represents a significant operational gap in large-scale infrastructure management.

This project directly targets this gap by building a system that learns normal log sequence behaviour from historical data and automatically flags deviations that correspond to actual system faults, using the well-established BGL supercomputer log benchmark to provide rigorous, reproducible evaluation.

1d. Research Questions

RQ1: To what extent can machine learning models trained on parsed BGL supercomputer log sequences outperform rule-based baseline anomaly detection approaches in terms of Precision, Recall, F1-score, and AUC-ROC on unseen log data?

RQ2: Which log event features and temporal sequence characteristics most significantly influence anomaly detection accuracy, as identified through SHAP feature importance attribution across candidate model architectures?

RQ3: How does the log parsing strategy (template-based versus raw token-based feature extraction) affect the downstream anomaly detection performance of machine learning models trained on the BGL dataset?

1e. Research Objectives

O1 (Data Curation): Acquire, parse, and preprocess the BGL supercomputer log dataset from the LogHub repository into cleaned, structured, time-series-ready feature matrices, including log parsing using the Drain algorithm, handling of missing entries, and label alignment for supervised and unsupervised modelling.

O2 (Feature Engineering): Engineer a comprehensive log feature set incorporating log event count vectors, TF-IDF-weighted event frequency matrices, temporal sliding window sequences, log level severity encodings, and component identifier embeddings to capture both static and dynamic anomaly signals.

O3 (Architecture Design): Design and implement four candidate anomaly detection model architectures — Logistic Regression (baseline), Random Forest, Isolation Forest (unsupervised), and LSTM Autoencoder — covering supervised, unsupervised, and deep learning paradigms.

O4 (Model Training and Evaluation): Train all candidate models using stratified cross-validation with appropriate handling of class imbalance via SMOTE oversampling, evaluate on held-out test sets, and apply SHAP analysis to identify the most influential log event features driving anomaly predictions.

O5 (Evaluation): Benchmark all models against a rule-based keyword-matching baseline using Precision, Recall, F1-score, AUC-ROC, and detection latency, with explicit analysis of false positive and false negative rates.

O6 (Deployment): Deploy the system as a containerised application with a real-time log monitoring dashboard presenting anomaly alerts, confidence scores, SHAP feature importance visualisations, and historical detection accuracy metrics.

2. Research Area (Literature Review)

Log-based anomaly detection has emerged as a critical research area in systems reliability engineering, driven by the exponential growth of distributed system complexity and log volume. Early approaches to log analysis relied exclusively on keyword matching and regular expression-based pattern alerting, which remain prevalent in commercial monitoring tools but suffer from severe limitations in adaptability and recall on novel failure patterns.

He et al. (2016) conducted a landmark study on automated log parsing, introducing the Drain algorithm as a highly efficient online log parsing approach capable of extracting structured event templates from raw unstructured log messages at production scale. Drain's O(n) parsing complexity and demonstrated accuracy across diverse log formats — including BGL, HDFS, and Windows event logs — establish it as the standard preprocessing foundation for machine learning-based log analysis pipelines, and it is adopted as the primary log parsing component in this project.

Du et al. (2017) introduced DeepLog, a seminal deep learning model for log anomaly detection that applies LSTM networks to model normal log sequences as execution workflows, flagging deviations from learned patterns as anomalies. Their evaluation on the BGL dataset demonstrated significant improvements over keyword-based baselines, establishing LSTM sequence modelling as a foundational technique for temporal log anomaly detection. The LSTM Autoencoder architecture in this project extends this approach by modelling anomalies as reconstruction errors rather than next-event prediction failures, enabling unsupervised anomaly scoring without requiring labelled training data.

Zhang et al. (2019) proposed Robust Log-based Anomaly Detection (RobustLog), which addresses the challenge of semantic drift in log formats over time. Their work highlights a critical practical limitation of fixed-vocabulary approaches: as software systems evolve, log message formats change, causing model performance degradation. This project mitigates this risk by incorporating Drain's template-based parsing, which is more robust to minor log format variations than raw token matching.

Breunig et al. (2000) introduced the Local Outlier Factor (LOF) algorithm as an early density-based anomaly detection method, which along with Liu et al.'s (2008) Isolation Forest — a tree-based unsupervised anomaly detection algorithm — provides the unsupervised baseline comparators in this project. Isolation Forest's demonstrated effectiveness on high-dimensional tabular anomaly detection tasks and its computational efficiency on large-scale log feature matrices make it a strong unsupervised candidate for comparison against supervised classifiers.

Zhu et al. (2019) published a systematic comparison of log analysis techniques on the LogHub benchmark suite, which includes the BGL dataset. Their evaluation established F1-score benchmarks for multiple state-of-the-art methods and identified the challenges of class imbalance, log parsing accuracy, and feature representation choice as the primary determinants of anomaly detection performance. These findings directly inform the experimental design choices made in this proposal, particularly the adoption of SMOTE for class imbalance handling and the comparative evaluation of multiple feature representation strategies.

Regarding explainability in anomaly detection, Lundberg and Lee (2017) introduced SHAP values as a unified framework for explaining machine learning model predictions across diverse model types including tree ensembles and neural networks. The integration of SHAP into log anomaly detection systems remains underexplored in published literature, with most studies reporting model performance metrics without providing operators actionable explanations of which log events triggered anomaly alerts. This project explicitly addresses this gap by integrating SHAP explanations into the operational monitoring dashboard.

2a. Research Gap

Despite substantial progress in log anomaly detection research, three specific gaps remain underaddressed. First, most published log anomaly detection studies report model performance without integrating interpretability tools into the operational system itself; the deployment of SHAP-based per-alert explanations within a live monitoring dashboard represents an underexplored practical contribution. Second, comparative evaluations that systematically assess both supervised (Random Forest, Logistic Regression) and unsupervised (Isolation Forest, LSTM Autoencoder) approaches on the BGL dataset within a single unified pipeline and feature engineering framework

are sparse, limiting evidence-based model selection for practitioners. Third, existing studies rarely evaluate detection latency alongside classification metrics, despite latency being a critical operational requirement for real-time fault detection systems. This project addresses all three gaps through a fully integrated, deployable system with multi-paradigm model comparison, operational SHAP explainability, and explicit latency benchmarking.

2b. Search Strategy

The literature review was conducted using three primary academic databases: Google Scholar, IEEE Xplore, and ACM Digital Library. arXiv was used as a supplementary source for recent preprints. The following keyword combinations guided the search: "log anomaly detection machine learning", "LSTM log analysis", "log parsing Drain algorithm", "BGL dataset anomaly detection", "Isolation Forest log data", "SHAP explainability anomaly detection", "distributed systems log monitoring", "DeepLog LSTM sequence", and "LogHub benchmark evaluation". Results were filtered to publications from 2000 onwards, with priority given to IEEE and ACM conference papers, journal articles, and highly cited arXiv preprints. Papers were assessed for relevance based on their focus on: (a) log parsing and preprocessing, (b) anomaly detection algorithms, (c) BGL or LogHub dataset evaluations, or (d) explainability in monitoring systems.

3. Expected Practical Element Output

The primary deliverable is a fully functional, deployable big data log anomaly detection system. The system comprises three main components: a log ingestion and preprocessing pipeline, a machine learning anomaly detection and scoring API, and an operator-facing real-time monitoring dashboard.

The data pipeline ingests raw log data from the BGL supercomputer log dataset (https://github.com/logpai/loghub/tree/master/BGL), which contains over four million log messages with binary alert labels. Processing steps include: applying the Drain log parsing algorithm to extract structured event templates from raw unstructured log messages; constructing sliding window feature matrices capturing log event sequences over configurable time windows; computing TF-IDF-weighted event count vectors; encoding log severity levels and component identifiers as numerical features; handling class imbalance via SMOTE oversampling on training splits; and performing exploratory data analysis to characterise event frequency distributions, anomaly class ratios, and temporal clustering patterns.

The machine learning component implements four candidate anomaly detection architectures: Logistic Regression and keyword-matching rule-based (baselines); Random Forest Classifier (supervised ensemble); Isolation Forest (unsupervised tree-based); and LSTM Autoencoder (deep learning sequence reconstruction). SHAP values are computed at inference time for supervised models to provide per-alert feature attribution. The best-performing model is served via a Flask REST API accepting log stream payloads and returning anomaly scores, binary predictions, and feature importance rankings.

The monitoring dashboard presents: a real-time log event stream with colour-coded anomaly alerts; per-alert SHAP visualisations identifying the specific log events contributing to each anomaly decision; model performance metrics including rolling Precision, Recall, and F1-score; and historical anomaly timelines enabling operators to identify recurring failure patterns.

Final deliverables include complete source code on GitHub, Jupyter notebooks documenting the full EDA, feature engineering, and model comparison workflow, a Docker deployment guide, and a comparative performance report benchmarking all models against rule-based baselines.

4. Required Resources

No proprietary hardware or paid software licences are required. All tools are open-source or freely available under academic licences.

Software stack: Python 3.10+ with libraries including Pandas, NumPy, Scikit-learn, PyTorch/TensorFlow (LSTM Autoencoder), SHAP, Drain3 (log parsing), Imbalanced-learn (SMOTE), Flask, Plotly, Matplotlib, and Jupyter Notebook. Git and GitHub for version control. Docker for containerisation.

Datasets are sourced entirely from the LogHub public repository at no cost:

- BGL (Blue Gene/L) Log Dataset — LogHub Repository: https://github.com/logpai/loghub/tree/master/BGL
- LogHub Benchmark Suite: https://github.com/logpai/loghub

Cloud infrastructure: AWS EC2 for model training and hosting, S3 for dataset and model artefact storage. Google Colab serves as an alternative free GPU environment for LSTM Autoencoder training. Estimated cloud costs are between £100 and £250 for the project duration.

Hardware: A minimum of 16GB RAM and a quad-core CPU is recommended for local development. GPU acceleration via Google Colab is recommended for LSTM Autoencoder training. Random Forest and Isolation Forest training can be completed on standard CPU infrastructure within acceptable timeframes.

5. Prerequisite Knowledge / Skills Required

Python Programming: Proficiency in Pandas, NumPy, Scikit-learn, PyTorch or TensorFlow, SHAP, Drain3, and Flask for end-to-end log analysis pipelines including preprocessing, model training, inference serving, and results visualisation.

Machine Learning and Anomaly Detection: Solid understanding of supervised classification algorithms, unsupervised anomaly detection methods (Isolation Forest), recurrent neural network architectures (LSTM, Autoencoders), evaluation metrics for imbalanced classification (F1-score, AUC-ROC, Precision-Recall curves), cross-validation strategies, and SMOTE-based class imbalance handling.

Log Analysis and Systems Knowledge: Understanding of distributed system log formats, log severity levels, system fault taxonomy, and the structural characteristics of supercomputer and cluster log data. Familiarity with log parsing tools including Drain and regular expression-based parsing. Awareness of common failure modes in large-scale computing infrastructure.

Web Development: Ability to build monitoring dashboards using HTML, CSS, JavaScript, and Plotly/Chart.js charting libraries, and REST APIs using Flask. Knowledge of real-time data streaming via WebSockets or Server-Sent Events for live log monitoring.

Statistics and Explainability: Understanding of feature importance methods, SHAP value computation for tree ensembles and neural networks, reconstruction error thresholding for autoencoder-based anomaly scoring, and statistical methods for anomaly threshold calibration on imbalanced datasets.

DevOps and Deployment: Familiarity with Docker containerisation, AWS EC2 and S3 for deployment and storage, GitHub Actions for CI/CD, and Agile project management for milestone delivery.

6. Project Plan — Gantt Chart

Timeline: 1 March 2025 — 10 May 2025 | Shaded cells = active work period

Task	Mar W1	Mar W2	Mar W3	Mar W4	Apr W1	Apr W2	Apr W3	Apr W4	May W1	May W2
1. Literature Review & Dataset Setup	■	■								
2. Data Preprocessing & EDA	■	■	■							
3. Feature Engineering		■	■	■						
4. Model Development & Training			■	■	■					
5. Model Evaluation & Comparison					■	■				
6. Web/Dashboard Development				■	■	■	■			
7. Integration & Testing						■	■	■		
8. Documentation & Report Writing							■	■	■	
9. Final Review & Submission									■	■

7. Data Sample Size Justification (Power Analysis)

The BGL (Blue Gene/L) dataset from the LogHub repository contains approximately 4,747,963 log messages collected over 214 days of operation from IBM's Blue Gene/L supercomputer system at Lawrence Livermore National Laboratory. Of these messages, 348,460 are labelled as alert messages (anomalous), representing a class imbalance ratio of approximately 1:12.6. This scale of labelled log data provides exceptional statistical power for both supervised and unsupervised model training.

For supervised classification tasks, a power analysis using G*Power (f^2 effect size = 0.15, $\alpha = 0.05$, power = 0.80, 25 predictor variables representing the log event feature set) yields a minimum required sample size of approximately 200 observations. The BGL dataset exceeds this minimum by a factor exceeding 23,000, providing overwhelming statistical power for detecting moderate effect sizes across all evaluation metrics.

For LSTM Autoencoder training, published guidelines recommend a minimum of 5,000–10,000 training sequences for stable convergence in sequence-to-sequence reconstruction tasks. Using a sliding window of 20 log events with a stride of 10, the BGL dataset yields approximately 470,000 training windows, far exceeding this minimum. A 70/15/15 stratified train/validation/test split will be applied, yielding approximately 330,000 training, 71,000 validation, and 71,000 test samples — all substantially above robustness thresholds for all candidate model architectures.

8. Ethical Considerations

Data Privacy and GDPR Compliance: The BGL log dataset is a publicly released, anonymised collection of supercomputer system telemetry data made available through the LogHub research repository. The dataset contains no personally identifiable information; it consists exclusively of system-generated log messages capturing hardware events, software states, and component identifiers from a decommissioned supercomputer system. No data relating to individual users or human subjects is present. The project does not fall within the scope of GDPR Article 4 data subject protections, and no ethical review for data privacy reasons is required, though a standard ethical self-assessment form will be completed as required by the programme.

Algorithmic Bias and Fairness: Anomaly detection models trained on BGL log data may not generalise to log formats and failure patterns from other system types (e.g., cloud microservices, web servers). This representational limitation will be explicitly acknowledged in the evaluation section. The system is designed as a decision support tool with human oversight; all anomaly alerts are presented to operators with confidence scores and SHAP explanations, ensuring human review of consequential incident response decisions and mitigating the risk of automated false-positive alerts triggering unnecessary operational interventions.

Responsible Deployment: The system will not be deployed on production live infrastructure during the project; evaluation is conducted exclusively on historical log data in a controlled research environment. All cloud resources will be terminated promptly after use to minimise unnecessary energy consumption. No sensitive credentials or API keys will be committed to the public GitHub repository; all secrets will be managed via environment variables and AWS IAM roles.

9. Risk Audit

Risk 1 — Dataset Accessibility (Likelihood: Low, Impact: Medium): The BGL dataset is hosted on GitHub's LogHub repository and is widely used in research; download disruption is unlikely. Mitigation: Download and verify dataset integrity during Week 1 and maintain a local backup copy.

Risk 2 — Log Parsing Accuracy (Likelihood: Medium, Impact: High): Drain log parsing may produce inaccurate event templates if the BGL log format contains unusual patterns, degrading downstream model feature quality. Mitigation: Validate parsing output against published BGL template benchmarks; manually inspect a random sample of 500 parsed messages; tune Drain's similarity threshold parameter.

Risk 3 — Class Imbalance Impact (Likelihood: Medium, Impact: High): The BGL dataset's 1:12.6 anomaly-to-normal class imbalance may cause models to exhibit high accuracy but low recall on the minority anomaly class. Mitigation: Apply SMOTE oversampling, use F1-score and AUC-ROC as primary evaluation metrics rather than accuracy, and tune classification thresholds on the validation set.

Risk 4 — LSTM Autoencoder Convergence (Likelihood: Medium, Impact: Medium): LSTM Autoencoders may fail to learn meaningful sequence representations, resulting in poor reconstruction error separation between normal and anomalous sequences. Mitigation: Apply dropout regularisation, early stopping, and gradient clipping; experiment with window sizes (10, 20, 50 events) to find optimal sequence granularity.

Risk 5 — Time Management (Likelihood: Medium, Impact: High): The concurrent demands of log parsing, multi-model implementation, dashboard development, and dissertation writing within the project timeline present scheduling risk. Mitigation: The Gantt chart allocates buffer time in Weeks 9–10; weekly supervisor check-ins provide early identification of slippage.

10. Limitations

Dataset Scope: This project relies exclusively on the BGL supercomputer log dataset. While BGL is a well-established LogHub benchmark, it represents a specific class of system (IBM Blue Gene/L supercomputer) from a single organisation. The log structure, failure taxonomy, and event distributions in BGL may differ significantly from modern cloud-native systems, microservice architectures, or web application logs. Findings should not be extrapolated as universally applicable without further validation across diverse log sources.

Offline Evaluation vs. Live Deployment: Due to time and resource constraints, the system will be evaluated in an offline batch processing mode using historical log replay rather than a live real-time log stream. Detection latency metrics will be estimated based on model inference times measured on representative test batches, rather than confirmed in a true streaming deployment.

Supervised Label Dependency: Supervised models (Logistic Regression, Random Forest) require labelled training data, which is available in the BGL benchmark but may be unavailable or expensive to produce in practice for novel systems. The Isolation Forest unsupervised component partially addresses this, but its performance may not match supervised models. Future work could explore semi-supervised and self-supervised approaches to reduce label dependency.

11. Impact and Contribution

Academic Contribution: This project contributes a rigorous, reproducible comparative evaluation of supervised, unsupervised, and deep learning log anomaly detection approaches on the BGL LogHub benchmark, incorporating SHAP-based interpretability as an integrated operational system component rather than a post-hoc analytical tool. The complete codebase, trained models, Jupyter notebooks, and evaluation artefacts will be made publicly available on GitHub under an open-source licence. The unified evaluation framework — comparing detection accuracy, false positive rates, and detection latency across all model paradigms — advances methodological rigour beyond single-model studies.

Industry Utility: The deployable system — a Dockerised Flask API with a real-time log monitoring dashboard — provides a practical proof-of-concept that SRE teams and infrastructure engineers can evaluate for integration into existing incident response workflows. The SHAP-based explainability layer directly addresses the operational challenge of "black box" anomaly alerts that provide no diagnostic guidance, offering engineers actionable, feature-level explanations of every anomaly detection decision. For organisations managing large-scale distributed systems, reducing mean time to detection (MTTD) of critical failures through automated anomaly alerting translates directly into improved service reliability and reduced incident impact.

References

Breunig, M. M., Kriegel, H., Ng, R. T. and Sander, J. (2000) 'LOF: Identifying Density-Based Local Outliers', in Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data. New York: ACM, pp. 93–104. doi:10.1145/342009.335388.

Du, M., Li, F., Zheng, G. and Srikumar, V. (2017) 'DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning', in Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS). New York: ACM, pp. 1285–1298. doi:10.1145/3133956.3134015.

He, P., Zhu, J., Zheng, Z. and Lyu, M. R. (2016) 'An Evaluation Study on Log Parsing and Its Use in Log Mining', in Proceedings of the 2016 IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). Toulouse: IEEE, pp. 654–661. doi:10.1109/DSN.2016.66.

Liu, F. T., Ting, K. M. and Zhou, Z.-H. (2008) 'Isolation Forest', in Proceedings of the 2008 Eighth IEEE International Conference on Data Mining (ICDM). Washington, DC: IEEE, pp. 413–422. doi:10.1109/ICDM.2008.17.

LogHub (2023) LogHub: A Large Collection of System Log Datasets for AI-driven Log Analytics. [Dataset Repository]. Available at: https://github.com/logpai/loghub (Accessed: February 2025). BGL Dataset: https://github.com/logpai/loghub/tree/master/BGL.

Lundberg, S. M. and Lee, S.-I. (2017) 'A Unified Approach to Interpreting Model Predictions', in Guyon, I. et al. (eds.) Advances in Neural Information Processing Systems 30 (NIPS 2017). Red Hook: Curran Associates, pp. 4765–4774. Available at: https://proceedings.neurips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html.

Zhang, X., Xu, Y., Lin, Q., Qiao, B., Zhang, H., Dang, Y., Xie, C., Yang, X., Cheng, Q., Li, Z., Chen, J., He, T., Yao, R., Lou, J.-G., Ding, M., Zhang, D. and Zhang, D. (2019) 'Robust Log-Based Anomaly Detection on Unstable Log Data', in Proceedings of the 2019 27th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE). New York: ACM, pp. 807–817. doi:10.1145/3338906.3338931.

Zhu, J., He, S., Liu, J., He, P., Xie, Q., Zheng, Z. and Lyu, M. R. (2019) 'Tools and Benchmarks for Automated Log Parsing', in Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP). Piscataway: IEEE Press, pp. 121–130. doi:10.1109/ICSE-SEIP.2019.00021.